3.3 DevOps

DevOps (Development + Operations) is a cultural philosophy, set of practices, and tools that automate and integrate software development and IT operations, breaking down silos to deliver software faster, more reliably, and with higher quality through collaboration, continuous feedback, and automation across the entire lifecycle (plan, code, build, test, release, deploy, operate, monitor).

It's about uniting people, processes, and technology to shorten the development cycle, increase deployment frequency, and provide better value to customers.

Core Principles & Culture

- **Collaboration:** Developers and operations work together, sharing responsibility.
- **Automation:** Automating repetitive tasks (testing, building, deploying) reduces errors and speeds things up.
- **Continuous Everything:** CI/CD (Continuous Integration/Continuous Delivery) for constant code integration, testing, and deployment.
- **Feedback Loops:** Constant monitoring and feedback to quickly adapt and improve.
- **Shared Tools:** Using common tools across the lifecycle.

Key Practices (The DevOps Lifecycle)

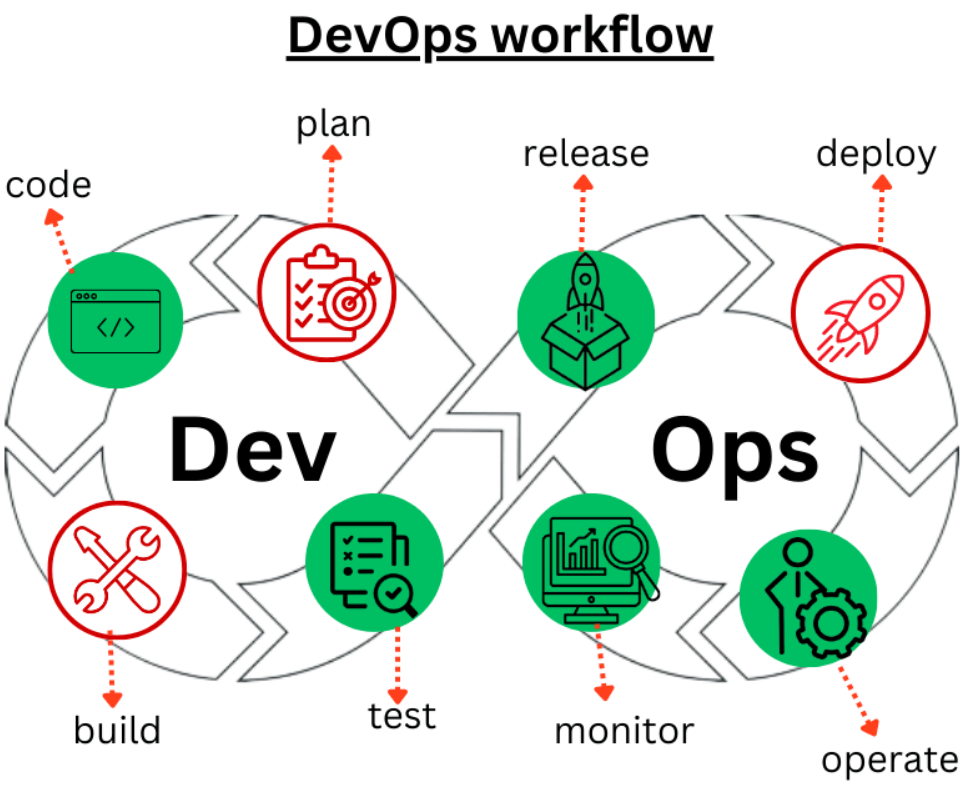
- **Plan:** Defining requirements and planning the work.
- **Code:** Writing the software.
- **Build:** Compiling code into deployable artifacts.
- **Test:** Automated testing for quality assurance.
- **Release/Deploy:** Getting the code into production.
- **Operate:** Running the application in production.
- **Monitor:** Tracking performance and issues in real-time.

Benefits

- Faster time-to-market for new features.
- Increased reliability and stability.
- Improved operational efficiency and productivity.
- Better ability to respond to customer needs.

DevOps Tools (Examples)

- **CI/CD:** Jenkins, GitLab CI, Azure Pipelines, GitHub Actions.
- **Containerization:** Docker, Kubernetes.
- **Monitoring:** Prometheus, Grafana, Sentry (error tracking).
- **Infrastructure as Code (IaC):** Terraform, Ansible.
- **Cloud Platforms:** Azure, AWS, GCP.



CI/CD (Continuous Integration/Continuous Delivery or Deployment)

CI/CD (Continuous Integration/Continuous Delivery or Deployment) is a DevOps practice that automates building, testing, and deploying software code, enabling faster, more reliable releases by integrating developer changes frequently into a shared repository and automating delivery to users.

It forms a **CI/CD pipeline**, a series of automated steps that reduce manual work, catch bugs early, and streamline the entire software development lifecycle (SDLC).

Typical Pipeline Stages

1. **Commit/Source:** Developer pushes code to a repository (e.g., Git).
2. **Build:** Code is compiled, and artifacts are created (e.g., Docker images).
3. **Test:** Automated tests (unit, integration, performance) run to find bugs.
4. **Deploy (Staging):** Application is deployed to a pre-production environment for further testing.
5. **Deploy (Production):** The final release to end-users (automated in Continuous Deployment).

Continuous Integration (CI)

- Developers frequently merge code changes into a central repository (e.g., multiple times a day).
- An automated system then builds and tests the code to quickly detect integration issues and bugs.

Continuous Delivery (CD)

- After successful CI, code changes are automatically prepared for release to a production-like environment.
- The final deployment to live customers often requires a manual approval step.

Continuous Deployment (CD)

- An extension of Continuous Delivery where every change that passes all automated tests is automatically deployed to production.
- This offers the fastest feedback loop and most frequent updates.

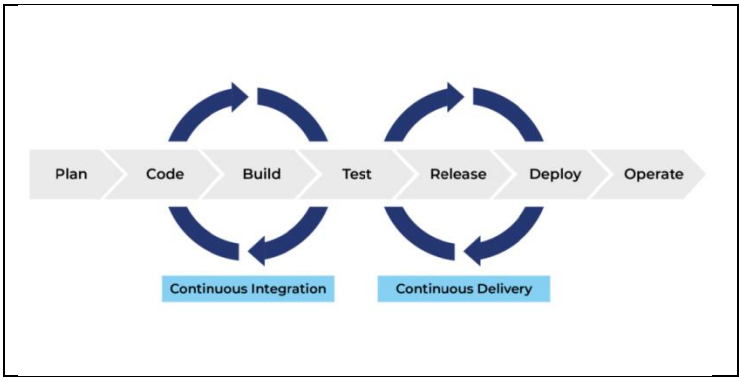
Key Benefits

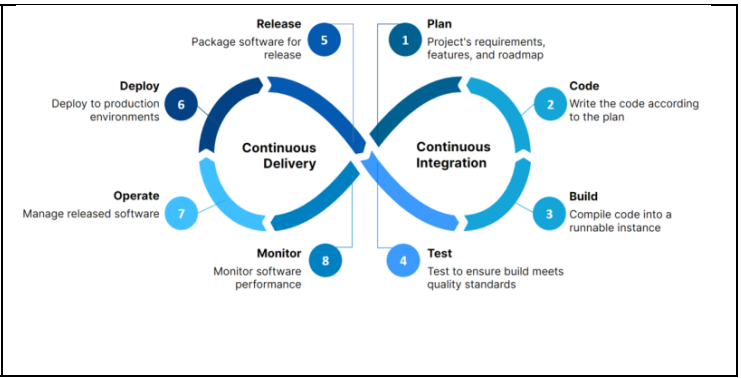
- **Faster Releases:** Automates slow, manual processes, allowing for more frequent updates.
- **Improved Quality:** Early detection of bugs reduces risk and cost.
- **Better Collaboration:** Bridges the gap between development and operations (DevOps).
- **Increased Efficiency:** Eliminates bottlenecks from manual tasks, freeing developers.

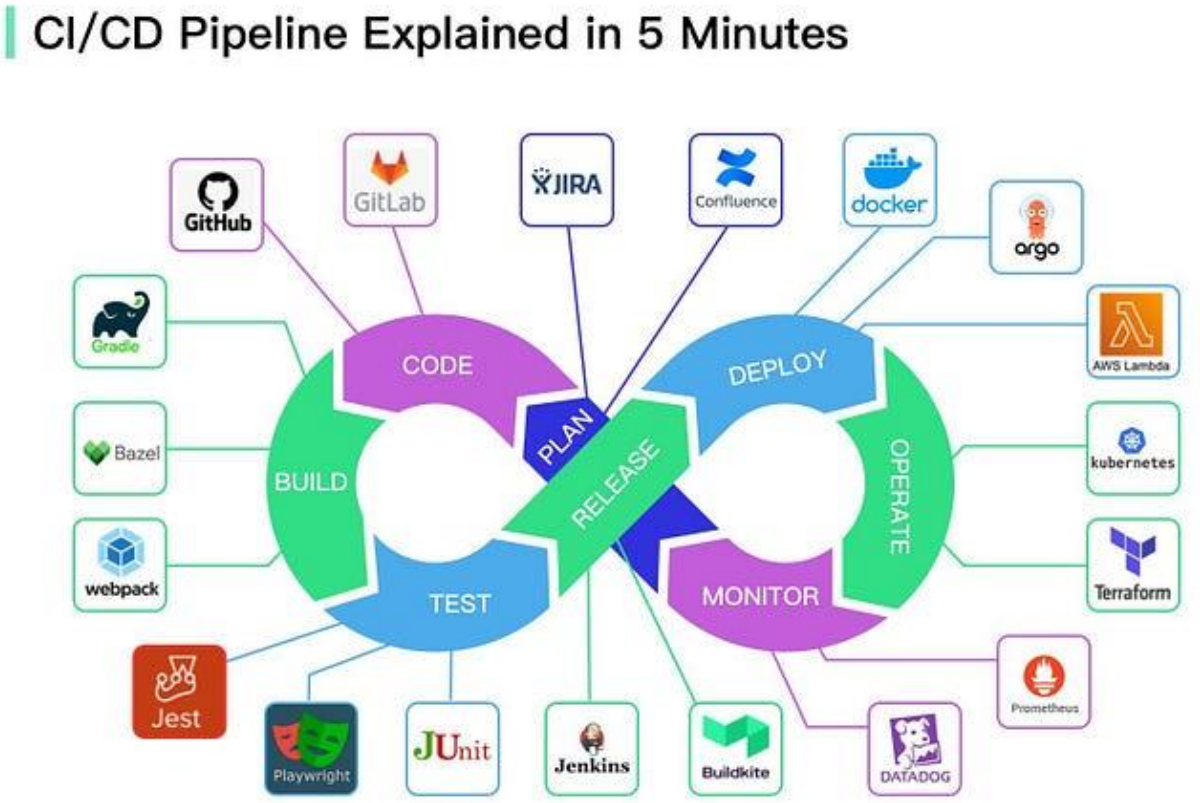
CI/CD pipeline

A CI/CD pipeline automates software delivery by creating a continuous flow from code commit to deployment, integrating Continuous Integration (CI) to build and test code changes frequently, and Continuous Delivery/Deployment (CD) to automatically release them, ensuring faster, more reliable, and higher-quality software with fewer manual errors.

This automated process includes stages like code commit, build, automated testing (unit, integration, security), and deployment to various environments (staging, production).







Pull Request in CI/CD pipelines

A **Pull Request (PR)** is a core component of modern software development workflows that integrates version control with **Continuous Integration/Continuous Deployment (CI/CD)** pipelines.

It acts as a formal proposal to merge code changes from a feature branch into a target branch (commonly `main` or `master`).

Role of a Pull Request in CI/CD

The primary function of a pull request in a CI/CD environment is to serve as a quality gate and collaboration hub before changes are integrated into the main codebase and potentially deployed.

- **Triggering Automated Checks**
- **Facilitating Code Review**
- **Preventing Conflicts**
- **Providing Traceability**
- **Enabling Preview Environments**

The Typical Pull Request Workflow in CI/CD

The process combines human collaboration with automated efficiency:

1. **Develop:** A developer creates a new feature branch from the main branch and commits their changes.
2. **Propose:** The developer opens a pull request via a platform like GitHub, GitLab, or Bitbucket, specifying the source and destination branches.
3. **Automate (CI):** The CI/CD system detects the new PR and automatically runs predefined jobs (tests, builds, scans).
4. **Review:** Team members review the code and the results of the automated checks.
5. **Iterate:** The developer pushes follow-up commits to the branch to address feedback, which automatically triggers re-runs of the CI checks.
6. **Merge & Deploy (CD):** Once approved and all checks pass, the PR is merged into the main branch. This merge triggers the CD pipeline, which deploys the changes to production or a staging environment.

Containerization

Containerization is a software deployment method that packages an application and all its dependencies (libraries, files, configurations) into a single, portable unit called a **container**, allowing it to run consistently and reliably across any infrastructure, from a developer's laptop to the cloud, by isolating it from the host operating system.

It's a lightweight form of virtualization, sharing the host's kernel but providing an isolated user space for the app, making it efficient, portable, and scalable, and is a cornerstone of modern cloud-native development, often using tools like Docker and Kubernetes.

How it Works

- **Packaging:** Code, runtime, system tools, libraries, and settings are bundled together.
- **Isolation:** Containers run in isolated environments but share the host operating system's kernel, unlike Virtual Machines (VMs) which each have a full OS.
- **Portability:** This self-contained package ensures the application behaves the same way everywhere, eliminating "it works on my machine" issues.

Key Benefits

- **Consistency:** Applications run identically in development, testing, and production.
- **Portability:** Easily move applications between different environments (on-premise, cloud, different OS).
- **Efficiency:** Lightweight and resource-efficient compared to VMs.
- **Speed:** Faster deployment and scaling due to standardized packaging.

Common Tools

- **Docker:** Popular platform for building, sharing, and running containers.
- **Kubernetes (K8s):** Orchestrates (manages, scales, deploys) containerized applications.

Deployment vs Implementation

Deployment is the technical act of making software or a system live (e.g., moving code to servers), while **Implementation** is the broader process of integrating it into business workflows, including training users and adapting processes to make it functional and valuable for the organization, with deployment being a key part of implementation. An example is implementing a new ERP: implementation involves process redesign and training, leading up to deployment (the "go-live" day) when the system is officially switched on for staff to use.

Example: New E-commerce Website

- **Implementation:**
  - Designing user roles (admin, customer).
  - Customizing features to fit business needs (e.g., specific product displays).
  - Integrating payment gateways and shipping APIs.
  - Training staff on how to manage products, process orders, and handle customer service.
  - Developing new workflows for order fulfillment.
- **Deployment:**
  - Uploading the website's code to cloud servers.
  - Setting up the database and content delivery network (CDN).
  - Configuring server settings and network rules.
  - Making the URL live and accessible to the public.

Key Differences

- **Scope:** Implementation is holistic (people, process, technology); deployment is technical (code, servers, infrastructure).
- **Focus:** Implementation is about *adoption and value*; deployment is about *availability and function*.
- **Timing:** Deployment is a specific event (the "go-live"); implementation is the entire journey, including pre-deployment setup and post-deployment support.